# 11.1 Gateway 的角色与设计目标

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~160,000 tokens，输出 ~12,000 tokens（本章合计）

从本章开始，我们正式进入 OpenClaw 最核心的子系统——Gateway 控制平面的源码分析。Gateway 是整个 OpenClaw 系统的心脏，搞懂它的设计思路和实现细节，才算真正理解了这个项目。

## 11.1.1 "控制平面"（Control Plane）的含义

> **衍生解释**：控制平面（Control Plane）是一个源自网络工程和分布式系统的概念。在计算机网络中，控制平面负责决策——决定数据包应该被转发到哪里；而数据平面（Data Plane）负责执行——实际转发数据包。类似地，在微服务架构中，控制平面（如 Kubernetes API Server）负责管理和协调，而各个工作节点负责实际执行任务。
>
> 在 OpenClaw 中，Gateway 就扮演控制平面的角色。它本身不执行 AI 推理（那是 Pi Agent 运行时的工作），也不直接处理消息格式转换（那是通道适配器的工作）。它做的是**协调**：接收来自各方的请求和事件，做出路由决策，调度执行，分发结果。

Gateway 的核心职责可以分为六大领域：

### 1. 连接管理

Gateway 维护两种不同性质的连接，理解它们的区别至关重要：

**通道（Channels）——嵌入式消息适配器**

通道**不是**通过网络连接到 Gateway 的外部服务，而是作为插件/适配器**嵌入在 Gateway 进程内部**。Gateway 启动时会调用 `startChannels()`（`src/gateway/server-channels.ts`）来初始化所有已配置的通道。通道与 Gateway 之间的通信是**进程内函数调用**。

每个通道负责与其对应的外部消息平台通信：

- **Telegram**：使用 grammY 库通过 HTTP 长轮询（或 Webhook）与 Telegram 服务器通信
- **WhatsApp**：使用 Baileys 库通过 WebSocket 与 WhatsApp 服务器通信
- **Discord**：使用 discord.js 库通过 WebSocket 与 Discord 网关通信

- **Slack**：使用 Bolt 库通过 WebSocket（Socket Mode）或 HTTP 事件与 Slack 通信

注意：这里的"通信"是指通道与**外部平台服务器**之间的网络通信。通道与 Gateway 之间**没有**网络通信——它们在同一个 Node.js 进程中，通过函数调用传递消息。

**客户端（Clients）——WebSocket 远程连接**

客户端是通过 WebSocket 协议连接到 Gateway 的**外部程序**，监听端口为 `18789` 。客户端分为两种角色：

- **操作者（operator）**：macOS 菜单栏应用、CLI 工具、Web 控制台。用于监控 Agent 运行状态、审批命令执行、管理会话和配置。
- **节点（node）**：macOS/iOS/Android 设备节点。用于向 Agent 暴露设备能力——摄像头、屏幕录制、位置信息、Canvas 画布等。

每个客户端连接都有身份认证、权限级别和事件订阅状态。Gateway 跟踪所有活跃连接，并在状态变更时向相关方推送事件。

> **关键区分**：在简单的消息问答场景中（如用户在 Telegram 上发消息、Agent 回复），客户端完全不参与。消息从通道进入 Gateway → 路由到 Agent → Agent 生成回复 → 回复通过通道发回用户，整个过程在 Gateway 进程内完成。客户端只在以下场景中参与：
>
> - Agent 需要执行危险命令，需要操作者审批（exec approval）
> - Agent 需要调用设备能力（如拍照、录屏），需要节点配合
> - 操作者主动查看 Agent 运行状态或管理会话

## 2. 消息路由

当一条消息从任何通道进来时，Gateway 需要决定：

- 这条消息应该交给哪个 Agent 处理？（多 Agent 路由）
- 使用哪个会话？（会话键生成）
- 该会话当前是否有运行中的 Agent？（队列管理）
- 回复应该发回哪个通道？（出站路由）

## 3. 会话管理

会话是 OpenClaw 中对话上下文的容器。Gateway 管理会话的完整生命周期：创建、读写、重置、过期、压缩、删除。

## 4. Agent 调度

当需要 AI 处理时，Gateway 启动 Agent 运行，管理队列、超时和取消。Agent 运行的结果通过事件流推送给所有感兴趣的客户端。

## 5. 工具基础设施

某些工具（如 Cron 调度、浏览器控制、节点调用）需要 Gateway 层面的支持。Gateway 提供这些工具所需的基础设施。

## 6. 配置与运维

Gateway 负责加载、校验、热重载配置，提供健康检查端点，管理守护进程生命周期。

# 11.1.2 单 Gateway 实例约束

OpenClaw 有一个重要的设计约束：**每台主机只运行一个 Gateway 实例**。这在 `docs/concepts/architecture.md` 中被明确为不变量（Invariant）：

> *"Exactly one Gateway controls a single Baileys session per host."*

这个约束来自 WhatsApp 的限制——Baileys（WhatsApp Web 协议的非官方实现）只允许一个活跃的 Web 会话。如果两个进程同时打开同一个 WhatsApp 账号的 Web 会话，前一个会被踢下线。

OpenClaw 通过文件锁和端口独占来强制这一约束。如果你尝试在同一台机器上启动第二个 Gateway，它会检测到端口已被占用并报错。相关实现在 `src/infra/ports.ts` 中：

```
// 概念示意
export class PortInUseError extends Error {
  constructor(public port: number, public owner?: string) {
    super(`Port ${port} is already in use${owner ? ` by ${owner}` : ""}`);
  }
}

export async function ensurePortAvailable(port: number): Promise<void> {
  // 尝试绑定端口，如果失败则抛出 PortInUseError
}
```

## 11.1.3 Gateway 的职责边界

搞清楚 Gateway **不做什么**同样重要：

| Gateway 做的 | Gateway 不做的 |
| --- | --- |
| WebSocket/HTTP 服务器管理 | AI 模型推理（交给 Pi Agent） |
| 消息路由与会话键解析 | 通道协议实现（交给通道适配器） |
| Agent 运行调度与队列管理 | 具体工具执行（交给工具系统） |
| 事件广播与客户端通知 | 原生应用 UI 渲染（交给客户端） |
| 配置加载与热重载 | 模型选择与 Auth Profile 管理（交给 Agent） |
| 安全认证与设备配对 | Markdown 渲染（交给通道出站处理） |

这种清晰的职责边界使得 Gateway 的代码虽然文件数量多（128 个文件），但每个文件的职责都比较单一。以 `src/gateway/` 下的文件命名为例：

```
server-channels.ts      → 管理通道连接
server-chat.ts          → 处理聊天消息/Agent 事件
server-cron.ts          → 管理 Cron 调度
server-http.ts          → HTTP 路由
server-ws-runtime.ts    → WebSocket 运行时
server-lanes.ts         → 队列车道
server-broadcast.ts     → 事件广播
server-discovery.ts     → 服务发现
server-tailscale.ts     → Tailscale 集成
server-maintenance.ts   → 定期维护任务
server-startup.ts       → 启动序列
server-close.ts         → 关闭序列
```

每个文件名都直观地反映了它的职责。这种命名约定贯穿了整个 OpenClaw 代码库。